

UNIVERSIDADE DO OESTE DE SANTA CATARINA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

MATEUS FERREIRA DA SILVA
RHYAN MEZAROBA
THIAGO THOMASI
THIAGO WURSTER BALBINOT

TRANSPORTES DOS GURI:
Sistema Gerenciador de Transporte Aéreo

Videira – SC
2026

1. Objetivos da Solução

Desenvolvimento de uma plataforma Web para gerenciamento de viagens corporativas e transporte executivo, permitindo o cadastro de clientes, agendamento de viagens, controle de aeronaves, voos e aeroportos, acompanhamento dos status das passagens, emissão de relatórios, integração com APIs externas para cadastrar CEP e controles de acesso, diferenciando painéis de clientes e administradores do sistema.

2. Tecnologias Utilizadas

2.1 Backend

- ASP .NET Core 9
- C#
- Entity Framework Core
- REST API

2.2 Frontend

- Razor Pages
- HTML5
- CSS3
- Tailwind CSS
- JavaScript

2.3 Banco de Dados

- SQL Server 2022
- PostgreSQL

2.4 Infraestrutura para Produção

- Servidor com ESXi
- Máquina Virtual com Debian - Aplicação
- Máquina Virtual com Debian - Banco de Dados
- NGINX - Proxy Reverso
- GitHub - Versionamento

3. Arquitetura Adotada

Foi utilizado principalmente a Arquitetura MVC (Model - View - Controller), e também a MVVM (Model - View - ViewModel), seguindo as seguintes divisões:

3.1 Models - gerencia a lógica de negócios, as regras do sistema, o estado da aplicação e a comunicação direta com o banco de dados

Aeronave.cs

Aeroporto.cs

ApplicationUser.cs

Assento.cs

Compra.cs

Escala.cs

Reserva.cs

Viagem.cs

Voo.cs

VooPoltrona.cs

3.2 Views - responsável por apresentar os dados na tela (seja em aplicações web, mobile ou desktop) e receber as interações do usuário

AdminAeronaves - Create, Delete, Details, Edit e Index

AdminAeroportos - Create, Delete, Details, Edit e Index

AdminHome - Index

AdminViagens - Create, Delete, Details, Edit e Index

AdminVoos - Create, Delete, Details, Edit e Index

Home - Index e Privacy

Viagens - ConfirmacaoCompra, Recibo, MinhasViagens, Details e Index

3.3 Controllers - recebe as requisições enviadas pelo usuário através da View, processa essas solicitações, busca ou envia dados para o Model

AdminAeronavesController.cs

AdminAeroportosController.cs

AdminHomeController.cs

AdminViagensController.cs

AdminVoosController.cs

ConsultaCEPController.cs

HomeController.cs

ViagensController.cs

3.4 ViewModels - retém o estado da tela, requisita dados do Model, prepara essas informações para exibição e se comunica diretamente com a View

AeronavesCompletaCreateViewModel.cs

AeronavesListViewModel.cs

AssentoCreateViewModel.cs

EscalaInsertViewModel.cs

ViagemCreateViewModel.cs

VooCompletoCreateViewModel.cs

CepViewModel.cs

4. Design Patterns

Podemos destacar o uso de Repository, Dependency Injection e Singleton.

4.1 Repository - Utilizado para abstrair o acesso aos Dados, garantindo um menor acoplamento, facilidade para testes e melhor manutenção.

```
public interface ISearchBarRepository
{
    public Task<List<Viagem>> Search(string searchString);
}
}
```

4.2 Dependency Injection - Utilizado para injeção de dependências dos serviços e repositório, garantindo um código mais desacoplado.

```
builder.Services.AddScoped<ISearchBarRepository,  
SearchBarRepository>();
```

4.3 MVC

Utilizado para separar responsabilidades entre interface, lógica de negócio e dados, promovendo maior organização do sistema.

5. Estrutura do Banco de Dados

Tirando as tabelas padrão do ASP .NET Core, temos 10 entidades alocadas em 10 tabelas diferentes no nosso Banco, podendo ser SQL Server ou PostgreSQL.

5.1 Aeronave

```
4 referências  
public class Aeronave  
{  
    [Key]  
    13 referências  
    public Guid Id { get; set; }  
  
    [Required(ErrorMessage = "O tipo da aeronave é obrigatório.")]  
    9 referências  
    public TipoAeronave Tipo { get; set; }  
  
    [Required(ErrorMessage = "O modelo da aeronave é obrigatório.")]  
    [StringLength(100, ErrorMessage = "O modelo deve ter no máximo 100 caracteres.")]  
    15 referências  
    public string Modelo { get; set; } = string.Empty;  
  
    12 referências  
    public ICollection<Assento> Assentos { get; set; } = new List<Assento>();  
  
    1 referência  
    public ICollection<Voo> Voos { get; set; } = new List<Voo>();  
}  
  
5 referências  
public enum TipoAeronave  
{  
    NaoEspecificado = 0,  
    AeronaveComercial,  
    AeronaveExecutiva,  
    AeronaveCargueira,  
    AeronaveRegional,  
    Jato  
}
```

5.2 Aeroporto

```
18 referências
public class Aeroporto
{
    [Key]
    13 referências
    public Guid Id { get; set; }

    [Required, StringLength(100)]
    42 referências
    public string NomeAeroporto { get; set; } = null!;

    [Required, StringLength(100)]
    44 referências
    public string CidadeAeroporto { get; set; } = null!;

    [Required, StringLength(100)]
    19 referências
    public string EstadoAeroporto { get; set; } = null!;

    [Required, StringLength(100)]
    10 referências
    public string Pais { get; set; } = null!;

    1 referência
    public ICollection<Voo> VoosDeOrigem { get; set; } = new List<Voo>();

    1 referência
    public ICollection<Voo> VoosDeDestino { get; set; } = new List<Voo>();

    0 referências
    public ICollection<Escala> Escalas { get; set; } = new List<Escala>();

    1 referência
    public ICollection<Viagem> ViagensDeOrigem { get; set; } = new List<Viagem>();
    1 referência
    public ICollection<Viagem> ViagensDeDestino { get; set; } = new List<Viagem>();
}
```

5.3 ApplicationUser

```
51 referências
public class ApplicationUser : IdentityUser<Guid>
{
    [PersonalData]
    [Required, StringLength(100)]
    3 referências
    public string Nome { get; set; } = null!;

    [PersonalData]
    [Required, StringLength(100)]
    3 referências
    public string Sobrenome { get; set; } = null!;

    [PersonalData]
    [Required]
    [StringLength(11, MinimumLength = 11)]
    3 referências
    public string CPF { get; set; } = null!;

    [PersonalData]
    3 referências
    public int Idade { get; set; }

    [PersonalData]
    4 referências
    public GeneroCliente Genero { get; set; }

    [PersonalData]
    3 referências
    public TipoCliente Tipo { get; set; }

    3 referências
    public bool IsAdmin { get; set; }

    [PersonalData]
    [Required]
    [StringLength(8)]
    2 referências
    public string CodigocEP { get; set; } = null!;

    [PersonalData]
    [StringLength(200)]
    2 referências
    public string? Logradouro { get; set; }
}
```

```

[PersonalData]
[Required(ErrorMessage = "O número é obrigatório.")]
[StringLength(10)]
2 referências
public string? Numero { get; set; }

[PersonalData]
2 referências
public string? Bairro { get; set; }

[PersonalData]
2 referências
public string? Cidade { get; set; }

[PersonalData]
2 referências
public string? Estado { get; set; }

[PersonalData]
[InverseProperty("Cliente")]
1 referência
public ICollection<Reserva> Reservas { get; set; } = new List<Reserva>();
}

7 referências
public enum GeneroCliente
{
    Masculino,
    Feminino
}

7 referências
public enum TipoCliente
{
    Preferencial,
    Normal
}

```

5.4 Assento

```

7 referências
public class Assento
{
    [Key]
    5 referências
    public Guid Id { get; set; }

    [ForeignKey("Aeronave")]
    3 referências
    public Guid IdAeronave { get; set; }

    [Required]
    [StringLength(5)]
    12 referências
    public string? NumeroAssento { get; set; }

    10 referências
    public ClasseAssento Classe { get; set; }

    6 referências
    public LocalizacaoPoltrona Localizacao { get; set; }

    5 referências
    public LadoPoltrona Lado { get; set; }

    0 referências
    public Aeronave Aeronave { get; set; } = null!;

    1 referência
    public ICollection<VooPoltrona> VooPoltronas { get; set; } = new List<VooPoltrona>();
}

11 referências
public enum ClasseAssento
{
    Economica,
    Executiva,
    Primeira
}

5 referências
public enum LocalizacaoPoltrona
{
    Janela,
    Corredor,
    Meio
}

5 referências
public enum LadoPoltrona
{
    Esquerda,
    Meio,
    Direita
}

```

5.5 Compra

```
13 referências
public class Compra
{
    [Key]
    20 referências
    public Guid Id { get; set; }

    [ForeignKey("Cliente")]
    7 referências
    public Guid IdCliente { get; set; }

    [ForeignKey("Viagem")]
    2 referências
    public Guid IdViagem { get; set; }

    6 referências
    public DateTime DataCompra { get; set; } = DateTime.Now;

    [Column(TypeName = "decimal(18, 2)")]
    6 referências
    public decimal PrecoTotal { get; set; }

    8 referências
    public StatusCompra Status { get; set; } = StatusCompra.Pendente;

    1 referência
    public ApplicationUser Cliente { get; set; } = null!;
    25 referências
    public Viagem Viagem { get; set; } = null!;

    [InverseProperty("Compra")]
    19 referências
    public ICollection<Reserva> Reservas { get; set; } = new List<Reserva>();
}

9 referências
public enum StatusCompra
{
    Pendente,
    Concluida,
    Cancelada
}
```

5.6 Escala

```
6 referências
public class Escala
{
    [Key]
    1 referência
    public Guid Id { get; set; }

    [ForeignKey("Voo")]
    1 referência
    public Guid IdVoo { get; set; }

    [ForeignKey("AeroportoEscala")]
    2 referências
    public Guid IdAeroportoEscala { get; set; }

    6 referências
    public DateTime HorarioChegada { get; set; }

    4 referências
    public DateTime HorarioSaida { get; set; }

    0 referências
    public Voo Voo { get; set; } = null!;

    5 referências
    public Aeroporto AeroportoEscala { get; set; } = null!;
}
```

5.7 Reserva

```
12 referências
public class Reserva
{
    [Key]
    4 referências
    public Guid Id { get; set; }

    [ForeignKey("Cliente")]
    2 referências
    public Guid IdCliente { get; set; }

    [ForeignKey("VooPoltrona")]
    2 referências
    public Guid IdVooPoltrona { get; set; }

    [ForeignKey("Compra")]
    2 referências
    public Guid IdCompra { get; set; }

    2 referências
    public DateTime DataReserva { get; set; } = DateTime.Now;

    5 referências
    public StatusReserva Status { get; set; }

    [Column(TypeName = "decimal(18, 2)")]
    3 referências
    public decimal PrecoReserva { get; set; }

    1 referência
    public ApplicationUser Cliente { get; set; } = null!;

    22 referências
    public VooPoltrona VooPoltrona { get; set; } = null!;

    1 referência
    public Compra Compra { get; set; } = null!;
}

4 referências
public enum StatusReserva
{
    Pendente,
    Confirmada,
    Cancelada
}
```

5.8 Viagem

```
public class Viagem
{
    [Key]
    public Guid Id { get; set; }

    [Required]
    [StringLength(100)]
    public string NomeViagem { get; set; } = string.Empty;

    [Required]
    public Guid IdOrigem { get; set; }

    [Required]
    public Guid IdDestino { get; set; }

    public DateTime DataPartida { get; set; }

    public DateTime DataChegada { get; set; }

    public decimal PrecoTotal { get; set; }

    [ForeignKey("IdOrigem")]
    [InverseProperty("ViagensDeOrigem")]
    public Aeroporto AeroportoOrigem { get; set; } = null!;

    [ForeignKey("IdDestino")]
    [InverseProperty("ViagensDeDestino")]
    public Aeroporto AeroportoDestino { get; set; } = null!;

    [InverseProperty("Viagem")]
    public ICollection<Voo> Voos { get; set; } = new List<Voo>();
}
```

5.9 Voo

```
public class Voo
{
    [Key]
    public Guid Id { get; set; }

    [ForeignKey("Aeronave")]
    public Guid IdAeronave { get; set; }

    [ForeignKey("AeroportoOrigem")]
    public Guid IdOrigem { get; set; }

    [ForeignKey("AeroportoDestino")]
    public Guid IdDestino { get; set; }

    23 referências
    public DateTime HorárioSaida { get; set; }
    15 referências
    public DateTime HorárioChegada { get; set; }

    11 referências
    public decimal PreçoBase { get; set; }

    21 referências
    public Aeronave Aeronave { get; set; } = null!;

    [InverseProperty("VoosDeOrigem")]
    27 referências
    public Aeroporto AeroportoOrigem { get; set; } = null!;

    [InverseProperty("VoosDeDestino")]
    27 referências
    public Aeroporto AeroportoDestino { get; set; } = null!;

    12 referências
    public ICollection<Escala> Escalas { get; set; } = new List<Escala>();

    6 referências
    public ICollection<VooPoltrona> VooPoltronas { get; set; } = new List<VooPoltrona>();

    [ForeignKey("Viagem")]
    10 referências
    public Guid? IdViagem { get; set; }

    3 referências
    public Viagem? Viagem { get; set; }
}
```

5.10 VooPoltrona

```
12 referências
public class VooPoltrona
{
    [Key]
    2 referências
    public Guid Id { get; set; }

    [ForeignKey("Voo")]
    3 referências
    public Guid IdVoo { get; set; }

    [ForeignKey("Assento")]
    5 referências
    public Guid IdPoltrona { get; set; }

    6 referências
    public StatusPoltrona Status { get; set; }

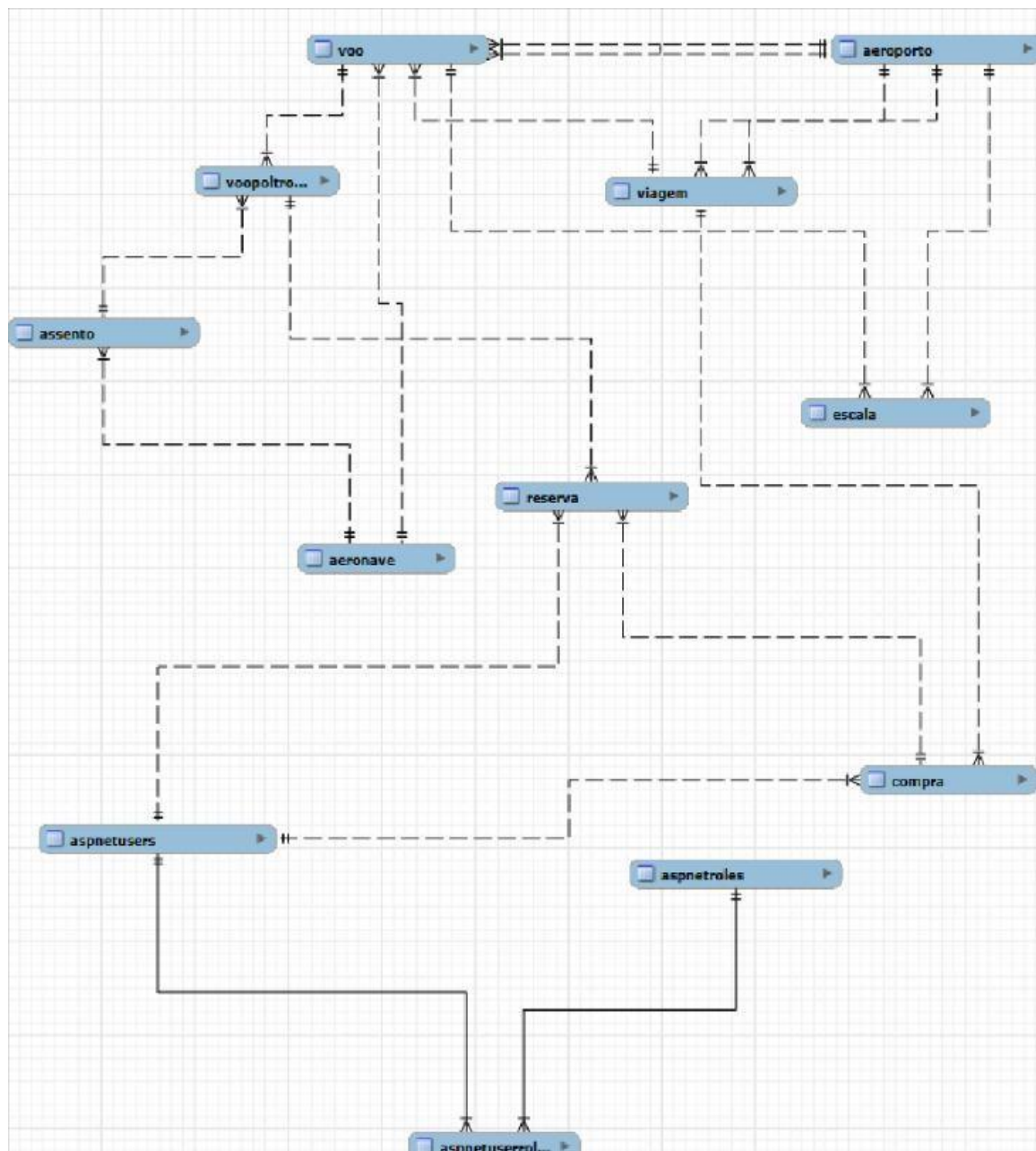
    20 referências
    public Voo Voo { get; set; } = null!;

    10 referências
    public Assento Assento { get; set; } = null!;

    1 referência
    public ICollection<Reserva> Reservas { get; set; } = new List<Reserva>();
}

7 referências
public enum StatusPoltrona
{
    Disponível,
    Reservado,
    Ocupado,
    Indisponível
}
```

6. Diagrama Simplificado

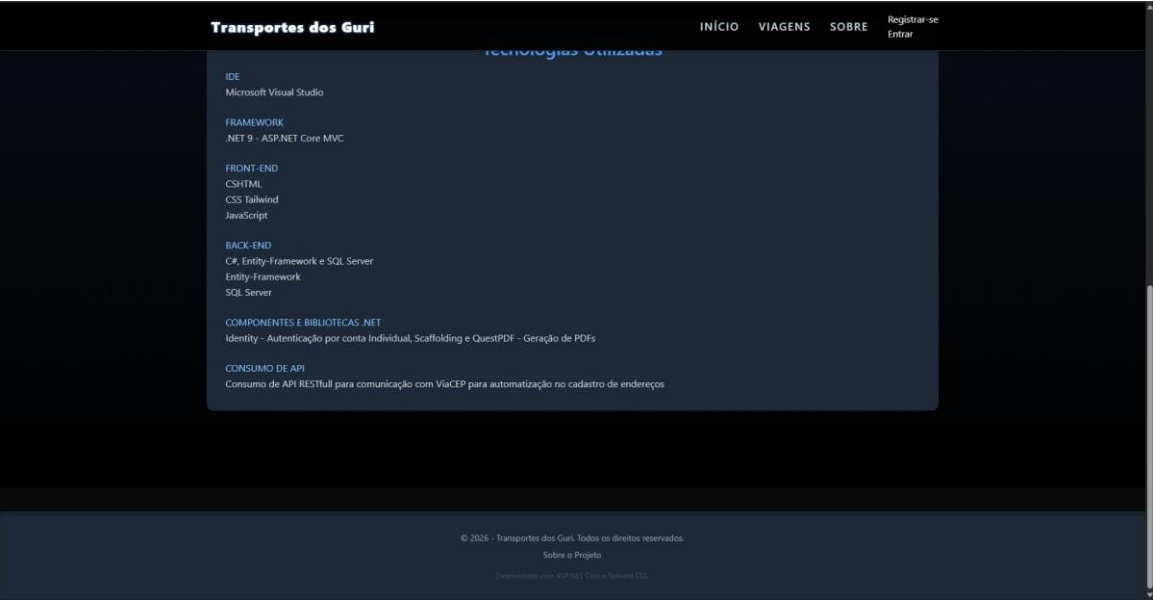


7. Prints da Aplicação

7.1 Home/Index:



7.2 Home/Privacy:



7.3 Identity/Account/Register

Transportes dos Guri

INÍCIOVIAGENS SOBRERegistrar-seEntrar

Criar uma Conta

Preencha os dados abaixo para registrar-se.

Nome

Informe seu Nome

Sobrenome

Informe seu Sobrenome

CPF

Informe seu CPF

Idade

Informe sua Idade

Gênero

-- Selecione o seu Gênero --

Tipo

-- Selecione a sua Classificação --

CEP

Ex: 89040000

Transportes dos Guri

INÍCIOVIAGENS SOBRERegistrar-seEntrar

CEP

Ex: 89645000

Consultar CEP

Logradouro

Rua / Avenida

Número

Número

Bairro

Bairro

Cidade

Cidade

Estado

UF

E-mail

nome@dominio.com

Senha

Senha

Confirmar Senha

Confirmar Senha

Administrador?

☐

Registrar

[Já possui conta? faça login](#)

7.4 Identity/Account/Login

Transportes dos Guri

INÍCIOVIAGENS SOBRERegistrar-seEntrar

Entrar

Use sua conta para acessar o sistema.

E-mail

conta@dominio.com

Senha

Senha

☒ Lembrar me

Entrar

[Registrar nova conta](#)

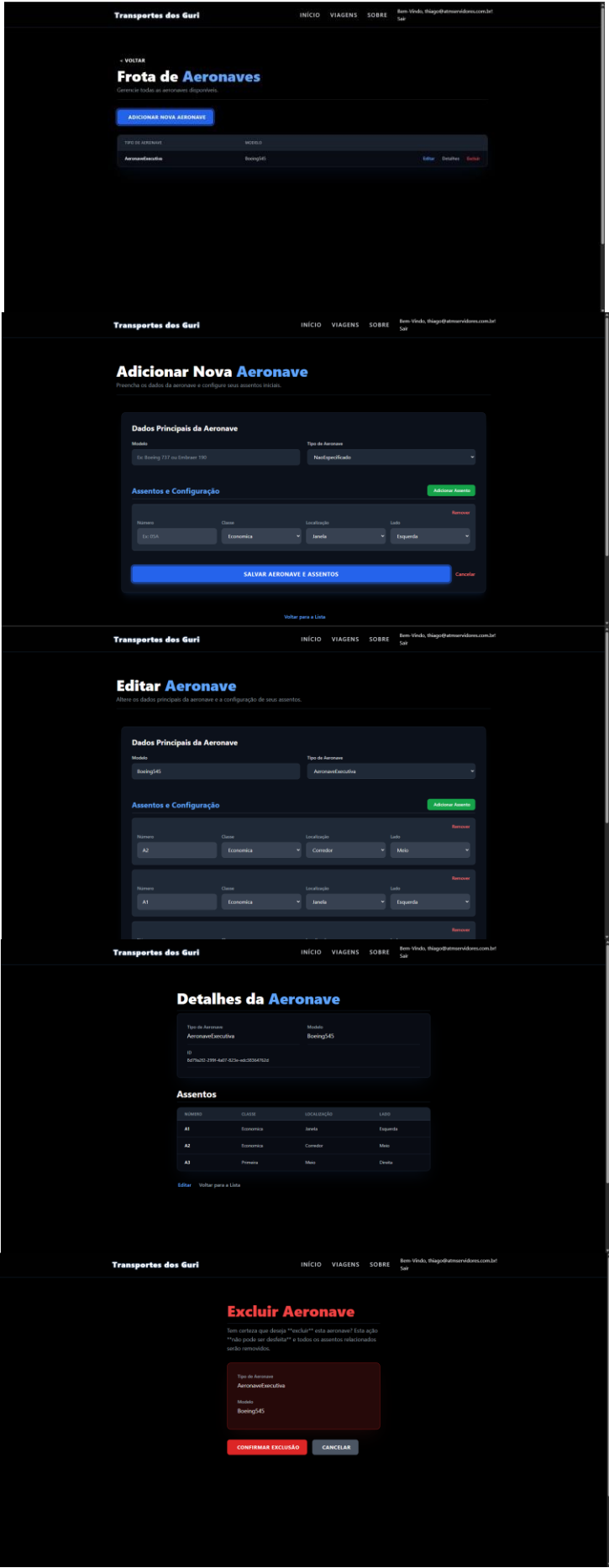
7.5 Identity/Account/Manage

The screenshot shows the 'Gerencie sua Conta' (Manage your account) page. The header includes the logo 'Transportes dos Guri' and navigation links: 'INÍCIO', 'VIAGENS', 'SOBRE', and a user profile section with 'Bem Vindo, thiballinot@gmail.com' and a 'Sair' (Logout) link. The main content area is titled 'Gerencie sua Conta' with a subtitle 'Ajuste suas configurações de perfil, segurança e dados pessoais.' Below this, there is a sidebar with links: 'Perfil', 'Senha', 'Recibos/Passagens', and 'Dados Pessoais'. The main form area is titled 'Perfil' and contains two input fields: 'thiballinot@gmail.com' and 'Insira seu telefone'. A blue 'Salvar' (Save) button is at the bottom of the form.

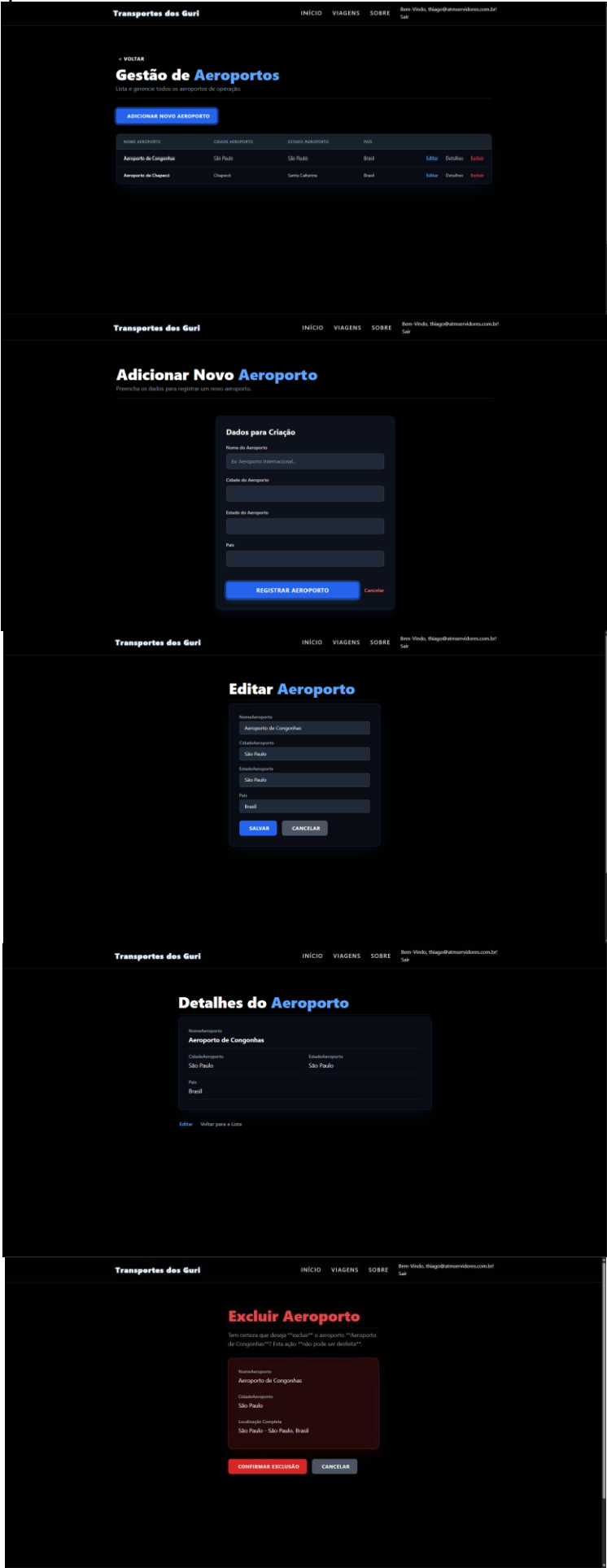
7.6 AdminHome

The screenshot shows the 'Painel Restrito de Administrador' (Admin Restricted Panel). The header is identical to the previous page, with the logo 'Transportes dos Guri' and navigation links: 'INÍCIO', 'VIAGENS', 'SOBRE', and a user profile section with 'Bem Vindo, thiago@atmservidores.com.br' and a 'Sair' (Logout) link. The main content area features a large title 'Painel Restrito de Administrador' and a subtitle 'Página Inicial para o gerenciamento de toda estrutura da Transportes dos Guri. Tecnologia de ponta para uma gestão otimizada.' Below the subtitle, there are four blue buttons arranged horizontally: 'GERENCIAMENTO DE AERONAVES', 'GERENCIAMENTO DE AEROPORTOS', 'GERENCIAMENTO DE VOOS', and 'GERENCIAMENTO DE VIAGENS'.

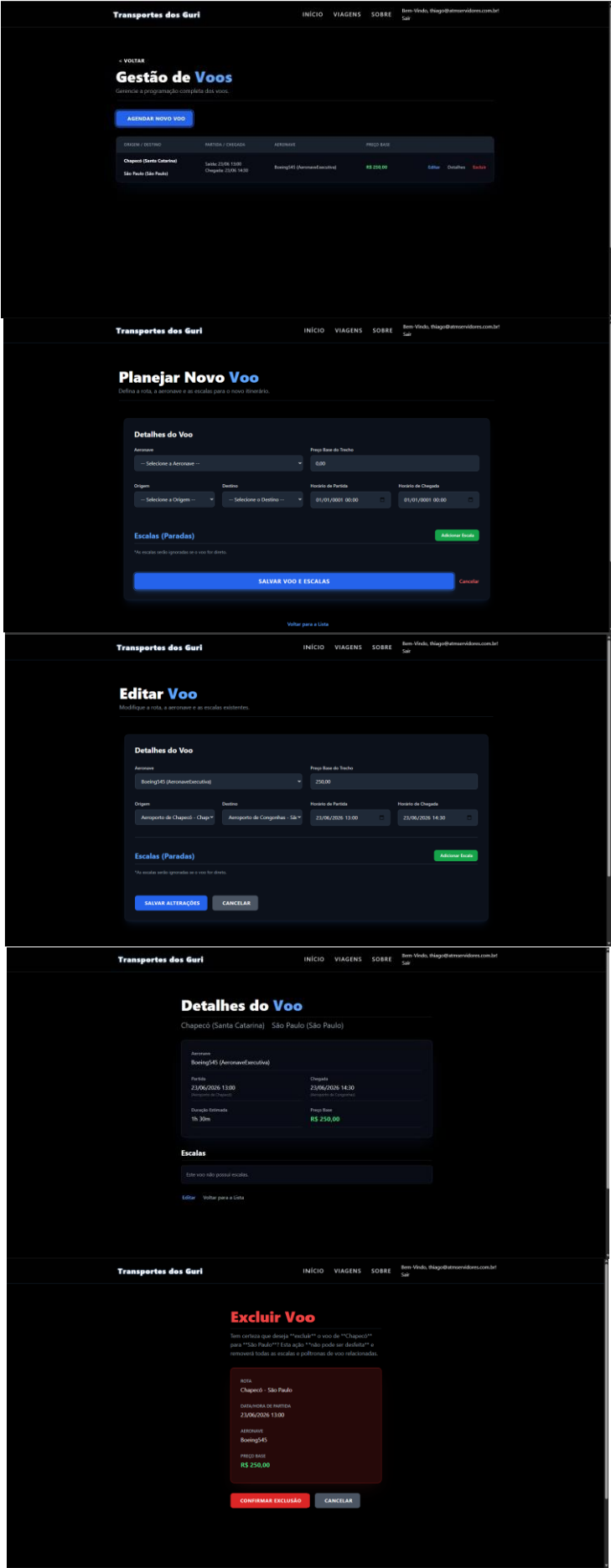
7.7 AdminAeronaves



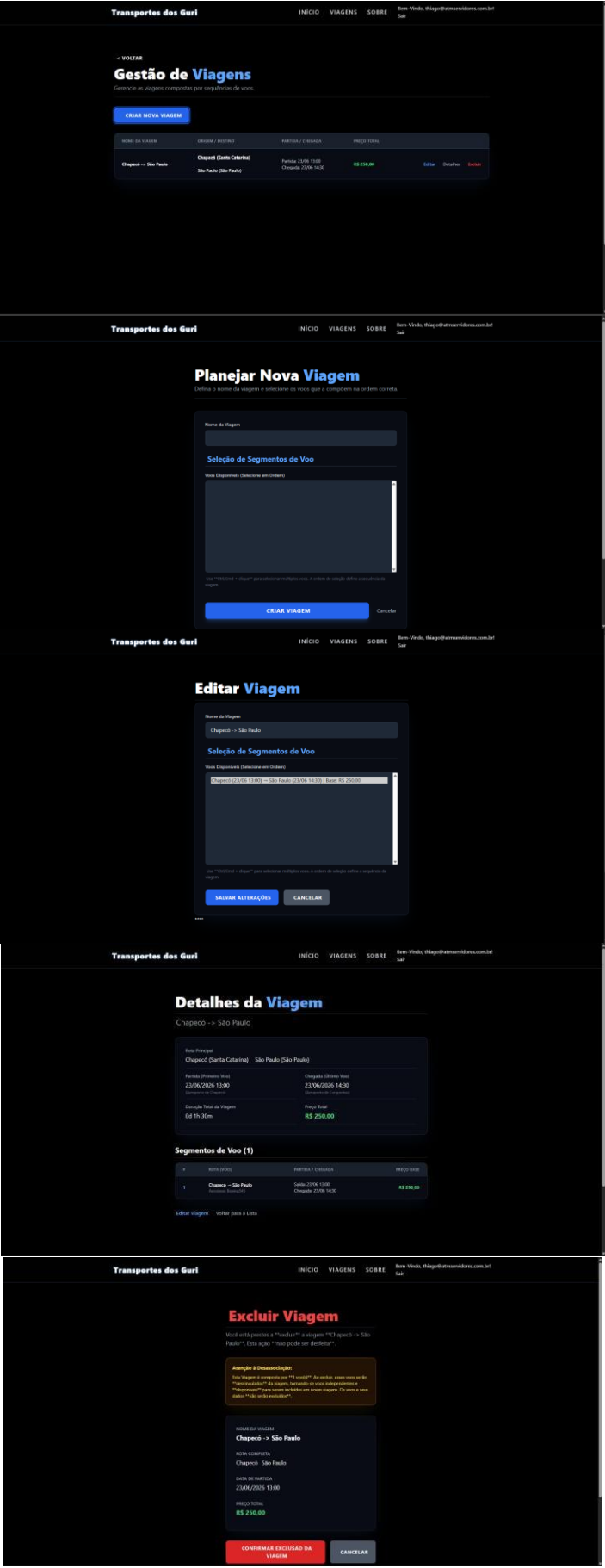
7.8 AdminAeroportos



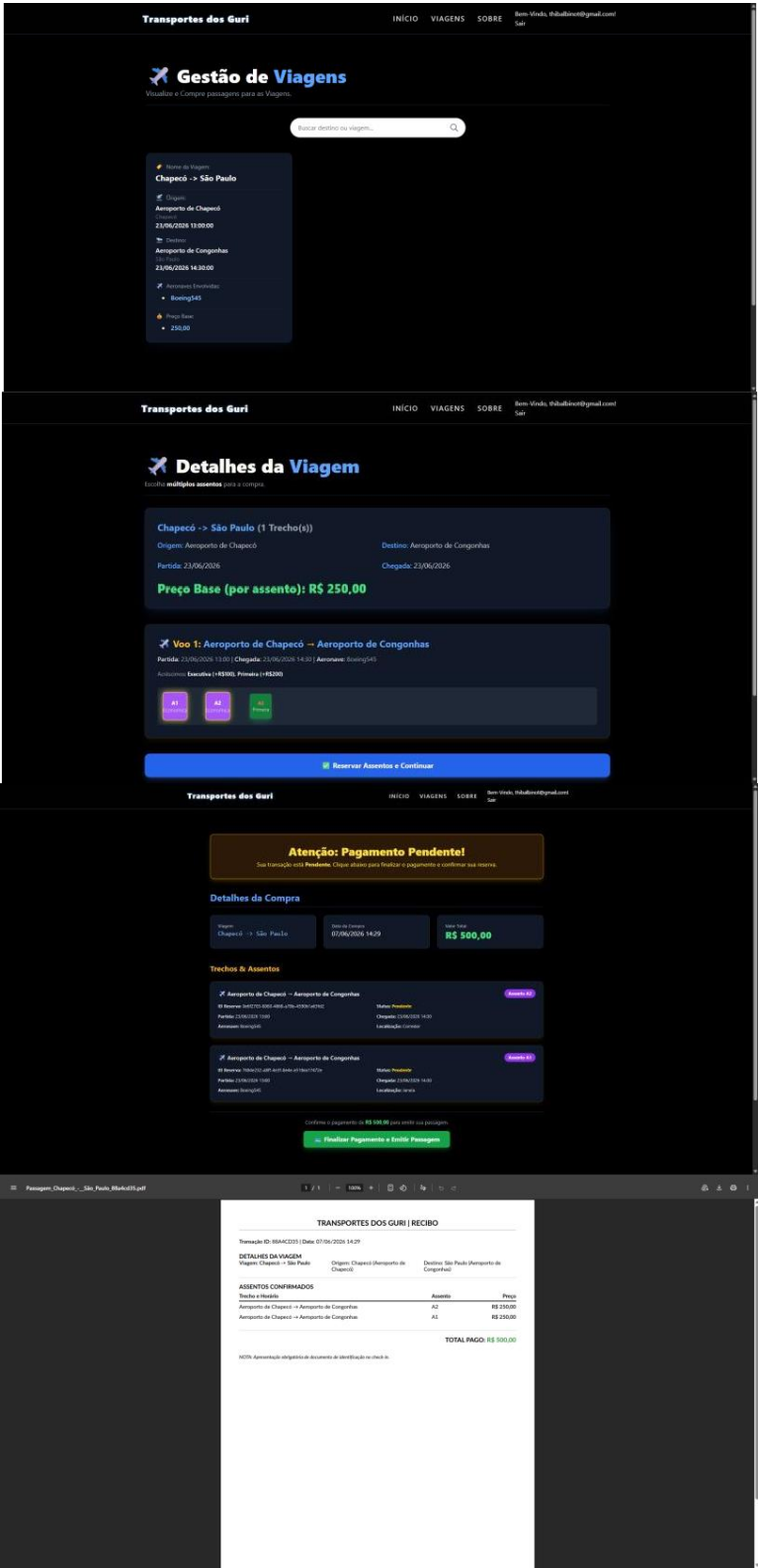
7.9 AdminVoos



7.10 AdminViagens



7.11 Viagens



8. Explicação do consumo de REST API

O controlador expõe um endpoint GET (api/ConsultaCEP) que recebe um CEP por parâmetro. Ele monta a URL do ViaCEP e usa a classe customizada ApiClient para disparar a requisição HTTP.

O ApiClient envia um HttpRequestMessage assíncrono via HttpClient e captura a resposta. Caso ocorra falha na rede ou erro HTTP (ex: 404), o cliente trata a exceção e retorna um JSON com mensagens de erro. Se a requisição falhar e o retorno contiver "errorMessages", o controlador devolve um BadRequest (400).

Se a requisição for bem-sucedida, o JSON retornado pelo ViaCEP é desserializado para a classe ViaCEP. Se a propriedade dto.cep vier nula (CEP inexistente na base), o controlador retorna NotFound (404). Caso contrário, os dados mapeados são transferidos para um objeto customizado da classe CEP. Por fim, o endpoint responde com status Ok (200), entregando ao cliente apenas os dados de endereço simplificados.

